

Guide de préparation à la soutenance PFA

Tastify – Digitalisation des services de restauration

Gestion des commandes, réservations et analyse de sentiment

Étudiants : Diouri Mehdi et El Kharrazi Ibtiha
Encadrante : Mme Bensaleh Nouhaila
Jury mentionné : M. Sraidi Khalid
Année universitaire : 2025–2026
Durée cible : 10 minutes maximum

Table des matières

1	Vue d'ensemble de la soutenance	2
2	Répartition entre les deux étudiants	3
3	Script oral détaillé slide par slide	5
4	Glossaire technique anti-questions pièges	9
5	Démonstration : deux scénarios	11
5.1	Scénario A : vidéo intégrée	11
5.2	Scénario B : démonstration en direct	11
6	Cinquante questions-réponses de jury	12
7	Questions personnelles pour chaque étudiant	13
8	Simulation de soutenance	14
9	Check-list de la veille et du jour J	14
10	Évaluation comme un jury	15

1 Vue d'ensemble de la soutenance

Sujet exact. Tastify est une application web full-stack de digitalisation des services de restauration. Elle centralise la gestion des commandes, des réservations, de la salle, du KDS, du stock, des paiements simulés, de la fidélité, des avis clients et de l'analyse de sentiment.

Objectif de la présentation. Montrer que le projet répond à un besoin métier concret : coordonner salle, cuisine, gestion et client dans un système cohérent, testable et démontrable.

Idée centrale à faire retenir. Tastify n'est pas seulement un menu numérique : c'est un prototype académique de mini-ERP restaurant qui relie les flux métier et exploite les avis clients.

Répartition de temps réaliste

#	Slide	Responsable	Durée	Objectif
1	TASTIFY	Ibtihal	15 s	Présenter sujet, équipe, encadrement.
2	PLAN DE LA PRÉSENTATION	Ibtihal	20 s	Annoncer le plan.
3	FLUX DE RESTAURATION	Ibtihal	25 s	Poser le contexte métier.
4	Problématique	Ibtihal	25 s	Formuler le besoin central.
5	Étude de l'existant	Ibtihal	30 s	Comparer avec des solutions commerciales.
6	Solution proposée	Ibtihal	35 s	Présenter les deux interfaces et les rôles.
7	Conception	Ibtihal	10 s	Ouvrir le bloc architecture et fonctionnement.
8	Architecture	Mehdi	35 s	Expliquer backend commun, API, WebSocket, Celery.
9	Commande vers KDS	Mehdi	30 s	Montrer le flux salle-cuisine.
10	Traitement avis	Mehdi	30 s	Expliquer l'asynchrone avec Celery.
11	Réalisation	Mehdi	10 s	Ouvrir le bloc technologies et IA.
12	ML et DL	Mehdi	25 s	Clarifier les notions.
13	Pipeline sentiment	Mehdi	40 s	Décrire données, routage, test, intégration.
14	Technologies	Mehdi	25 s	Justifier la stack réelle.
15	Modèles utilisés	Mehdi	30 s	Différencier BERT, MARBERT, fallback.
16	Comparaison performances	Mehdi	25 s	Présenter les résultats indicatifs.
17	Métriques pipeline	Mehdi	25 s	Expliquer precision, recall, F1.
18	Démonstration	Mehdi	45 s	Lancer ou commenter la démo courte.
19	Défis	Ibtihal	25 s	Montrer les difficultés maîtrisées.
20	Conclusion	Ibtihal	30 s	Bilan, limites, perspectives.

Si vous êtes en retard. Raccourcir d'abord les slides 5, 7, 11, 12 et 19. Ne jamais supprimer la problématique, l'architecture, l'analyse de sentiment, la démonstration courte, les limites et la conclusion.

2 Répartition entre les deux étudiants

Logique retenue. Ibtihal ouvre et ferme la soutenance : contexte, besoin, UX, solution, conclusion. Mehdi prend le bloc technique : architecture, backend, flux KDS, sentiment, modèles, métriques et démo technique. Cette répartition suit les contributions décrites dans le rapport.

#	Qui	Durée	Texte oral conseillé	Transition
1	Ibtihal	15 s	Bonjour, nous allons présenter Tastify, notre PFA sur la digitalisation des services de restauration.	Je commence par le plan de notre présentation.
2	Ibtihal	20 s	La présentation suit le parcours du projet : contexte, problématique, solution, conception, réalisation, analyse de sentiment, démonstration, défis et perspectives.	Avant la solution, il faut comprendre le flux réel d'un restaurant.
3	Ibtihal	25 s	Dans un restaurant, client, serveur, cuisinier et gérant échangent vite. Une information mal transmise peut créer une erreur, un retard ou une rupture.	C'est ce constat qui nous amène à la problématique.
4	Ibtihal	25 s	Notre question principale est de centraliser les opérations tout en améliorant la coordination entre salle, cuisine, gestion et clients.	Nous avons aussi regardé les solutions existantes.
5	Ibtihal	30 s	Les solutions existantes sont puissantes, mais peuvent être coûteuses ou complexes. Tastify est un prototype académique modulaire.	À partir de cela, voici notre solution.
6	Ibtihal	35 s	Tastify propose deux interfaces, un portail client et un back-office staff, reliés au même backend.	Je vais maintenant introduire la partie conception.
7	Ibtihal	10 s	Cette partie explique comment Tastify est organisé et comment les flux principaux fonctionnent.	Je laisse la parole à Mehdi pour l'architecture.
8	Mehdi	35 s	L'architecture repose sur un backend Django REST Framework, MySQL, Redis, Celery, Django Channels et deux frontends React.	Cette architecture sert notamment au flux de commande.

9	Mehdi	30 s	Le serveur crée une commande, le backend l'enregistre, puis la cuisine la reçoit dans le KDS via WebSocket.	Le même principe de traitement séparé existe pour les avis.
10	Mehdi	30 s	Un avis est sauvegardé rapidement, puis Celery lance l'analyse de sentiment en arrière-plan.	Nous passons maintenant à la partie réalisation.
11	Mehdi	10 s	Cette partie présente les technologies, le pipeline de sentiment et les résultats.	Je commence par clarifier ML et Deep Learning.
12	Mehdi	25 s	Le Machine Learning apprend à partir de données. Le Deep Learning utilise des réseaux de neurones profonds. BERT et MARBERT sont des modèles de Deep Learning utilisés via Hugging Face.	Voici le pipeline complet.
13	Mehdi	40 s	Le pipeline part du commentaire stocké en base, détecte simplement l'arabe, choisit le modèle, normalise la sortie, puis stocke le résultat.	La stack complète utilisée est la suivante.
14	Mehdi	25 s	Python, Django, DRF, React, TypeScript, MySQL, Redis, Celery, Channels, Hugging Face, pytest, Vitest, Playwright et Docker Compose forment la stack réelle.	Pour la partie IA, nous avons trois chemins.
15	Mehdi	30 s	Le modèle non arabe est NLP Town BERT multilingue, l'arabe passe vers MARBERT, et le fallback local sert de secours.	Nous avons comparé ces approches sur un petit corpus.
16	Mehdi	25 s	Sur 15 avis annotés manuellement, le fallback obtient 0,53 et le pipeline principal 0,93 d'accuracy. Ces chiffres restent indicatifs.	Regardons rapidement les métriques par classe.
17	Mehdi	25 s	La precision mesure la qualité des prédictions, le recall mesure les vrais cas retrouvés et le F1-score équilibre les deux.	Je vais maintenant lancer ou commenter la démonstration.
18	Mehdi	45 s	La démonstration montre le menu client, la commande staff, le KDS cuisine et les avis avec sentiment. Si la vidéo ne marche pas, je commente les écrans préparés.	Je rends la parole à Ibtihal pour les défis.
19	Ibtihal	25 s	Les défis principaux ont été le temps réel, l'asynchrone, les données multilingues et la validation.	Pour conclure, voici le bilan.

20	Ibtihal	30 s	Tastify centralise plusieurs modules restaurant et exploite les avis. Les perspectives sont le paiement réel, un corpus IA plus large, les tests de charge et le déploiement.	Nous terminons avec la slide de questions.
21	Ibtihal	10 s	Merci pour votre attention. Nous sommes prêts à répondre à vos questions.	–

3 Script oral détaillé slide par slide

Slide 1 – TASTIFY

Responsable : Ibtihal. **Durée max :** 25 s. **But :** installer le sujet et l'équipe.

À l'écran : titre Tastify, sujet, étudiants, encadrante, année, jury.

Script : Bonjour, nous allons présenter Tastify, notre projet de fin d'année. Le sujet porte sur la digitalisation des services de restauration : gestion des commandes, réservations et analyse de sentiment. Nous sommes Diouri Mehdi et El Kharrazi Ibtihal, encadrés par Mme Bensaleh Nouhaila.

Version courte : Bonjour, nous présentons Tastify, une application web pour centraliser les services d'un restaurant.

Mots techniques : digitalisation, analyse de sentiment.

Questions probables : Pourquoi ce sujet ? Réponse : parce qu'un restaurant combine plusieurs flux métier liés et faciles à démontrer.

Slide 2 – PLAN DE LA PRÉSENTATION

Responsable : Ibtihal. **Durée max :** 25 s. **But :** annoncer le plan.

Script : Nous allons avancer du besoin métier vers la solution : d'abord le contexte et la problématique, puis l'existant, la solution Tastify, la conception, la réalisation, le pipeline d'analyse de sentiment, les résultats, la démonstration, les défis et les perspectives.

Version courte : Le plan suit le projet : besoin, solution, conception, technique, IA, résultats et conclusion.

Question probable : Pourquoi cette progression ? Réponse : elle permet de comprendre le besoin avant les choix techniques.

Slide 3 – FLUX DE RESTAURATION

Responsable : Ibtihal. **Durée max :** 35 s.

Script : Un restaurant fonctionne avec un enchaînement rapide. Le client commande, le serveur transmet, la cuisine prépare, le gérant suit les stocks et les indicateurs. Quand ces informations sont séparées, on peut avoir des erreurs, des retards ou des ruptures.

Version courte : Le restaurant est un flux coordonné ; Tastify vise à réduire les ruptures d'information.

Question probable : Quels acteurs sont couverts ? Réponse : client, serveur, cuisinier et gérant.

Slide 4 – Problématique

Responsable : Ibtihal. **Durée max :** 35 s.

Script : Notre problématique est de centraliser les opérations d'un restaurant tout en améliorant la coordination entre la salle, la cuisine, la gestion et les clients. Nous voulions éviter une application limitée au menu, et construire un système plus cohérent.

Versión courte : Comment centraliser les opérations et fluidifier la coordination du restaurant ?

Question probable : Quelle est la différence avec une application de gestion classique ? Réponse : Tastify relie aussi le KDS, les avis, la fidélité et l'analyse de sentiment.

Slide 5 – Étude de l'existant

Responsable : Ibtihal. **Durée max :** 40 s.

Script : Les solutions existantes comme Lightspeed, Toast ou Revel sont puissantes. Leur limite, dans notre lecture, est le coût, la complexité ou l'adaptation au contexte local. Tastify n'essaie pas de les remplacer : c'est un prototype académique modulaire pour comprendre et implémenter les flux essentiels.

Versión courte : Les solutions commerciales valident le besoin, mais notre projet se concentre sur un prototype modulaire et pédagogique.

Question probable : Tastify est-il commercialisable ? Réponse : pas tel quel ; il faudrait renforcer production, sécurité, paiement réel et support.

Slide 6 – Solution proposée : Tastify

Responsable : Ibtihal. **Durée max :** 45 s.

Script : Tastify propose deux interfaces. Le portail client sert à consulter le menu, réserver, payer de manière simulée, suivre la fidélité et laisser un avis. Le back-office sert au gérant, serveur et cuisinier : salle, commandes, KDS, stocks, réservations, avis et tableau de bord.

Versión courte : Deux interfaces, un backend commun, quatre rôles.

Transition exacte : Je vais maintenant laisser la parole à Mehdi, qui va présenter l'architecture technique.

Question probable : Pourquoi deux frontends ? Réponse : les usages staff et client sont très différents.

Slide 7 – Conception

Responsable : Ibtihal. **Durée max :** 10 s.

Script : Cette slide ouvre la partie conception. Elle sert de transition : maintenant que le besoin et la solution sont posés, on explique comment Tastify est organisé techniquement.

Versión courte : Nous passons à l'architecture et au fonctionnement de Tastify.

Question probable : Pourquoi une slide de séparation ? Réponse : pour rendre la soutenance plus lisible et marquer le changement de bloc.

Slide 8 – Architecture de Tastify

Responsable : Mehdi. **Durée max :** 45 s.

Script : L'architecture est découplée. Les deux frontends React consomment le même backend Django REST Framework. MySQL stocke les données. Redis sert au channel layer WebSocket et au broker

Celery. Celery exécute les tâches de fond, par exemple l'analyse de sentiment.

Version courte : React côté interfaces, Django côté API, MySQL pour les données, Redis et Celery pour les flux non bloquants.

Question probable : Pourquoi WebSocket ? Réponse : pour mettre à jour le KDS et les événements staff sans recharger.

Slide 9 – Prise de commande et transmission au KDS

Responsable : Mehdi. **Durée max :** 40 s.

Script : Le serveur sélectionne une table et ajoute des plats. La commande est enregistrée côté backend, puis la cuisine la voit dans le KDS. Le cuisinier peut changer les statuts : en attente, en préparation, prêt ou servi selon le flux prévu.

Version courte : La commande part de la salle, passe par l'API, puis apparaît côté cuisine.

Question probable : Le serveur et le cuisinier ont-ils les mêmes droits ? Réponse : non, les rôles limitent les actions côté API.

Slide 10 – Traitement d'un avis client

Responsable : Mehdi. **Durée max :** 40 s.

Script : Quand un client laisse un avis, l'application enregistre le commentaire. Ensuite Celery lance l'analyse de sentiment en arrière-plan. Le résultat est stocké avec un label, un score et le modèle utilisé.

Version courte : L'avis est enregistré vite ; l'analyse se fait après, sans bloquer le client.

Question probable : Pourquoi asynchrone ? Réponse : pour ne pas dépendre du temps de réponse de Hugging Face dans l'expérience client.

Slide 11 – Réalisation

Responsable : Mehdi. **Durée max :** 10 s.

Script : Cette slide ouvre la partie réalisation. On passe des choix de conception à ce qui a été réellement implémenté : technologies, pipeline d'analyse de sentiment, modèles et résultats.

Version courte : Nous passons maintenant à la réalisation concrète.

Question probable : Que faut-il retenir ici ? Réponse : cette slide annonce le bloc technique implémenté.

Slide 12 – Machine Learning et Deep Learning

Responsable : Mehdi. **Durée max :** 35 s.

Script : Le Machine Learning apprend des règles à partir de données. Le Deep Learning est une sous-famille basée sur des réseaux de neurones profonds. Dans notre projet, BERT et MARBERT sont des modèles de Deep Learning déjà pré-entraînés et fine-tunés.

Version courte : Nous n'avons pas entraîné ces modèles ; nous les utilisons et les intégrons dans Tastify.

Question probable : Pourquoi parler de Deep Learning si c'est une API ? Réponse : l'API exécute des modèles de Deep Learning ; notre travail est leur intégration.

Slide 13 – Étapes du pipeline d'analyse de sentiment

Responsable : Mehdi. **Durée max :** 45 s.

Script : Le pipeline commence par le commentaire stocké en base MySQL. Le code détecte simplement les caractères arabes, limite le texte envoyé, choisit le modèle Hugging Face, normalise la sortie en positif, neutre ou négatif, puis stocke le résultat. La slide précise bien qu'il n'y a pas d'entraînement complet local : les modèles sont pré-entraînés et fine-tunés par leurs auteurs. Le test présenté repose sur un corpus manuel de 15 avis.

Versión courte : Données, prétraitement simple, choix du modèle, prédiction, stockage et exploitation.

Question probable : Quelle est la limite de la détection de langue ? Réponse : elle est simple et doit être améliorée pour les textes mixtes.

Slide 14 – Outils et technologies utilisés

Responsable : Mehdi. **Durée max :** 35 s.

Script : Nous utilisons Django et DRF pour le backend, React, TypeScript et Vite pour les interfaces, MySQL pour la base relationnelle, Redis et Celery pour les traitements non bloquants, Django Channels pour WebSocket, puis pytest, Vitest et Playwright pour les tests.

Versión courte : La stack est full-stack web, testée et lancée avec Docker Compose.

Question probable : Pourquoi Docker ? Réponse : pour lancer de manière reproductible tous les services de démo.

Slide 15 – Modèles utilisés pour l'analyse de sentiment

Responsable : Mehdi. **Durée max :** 40 s.

Script : Le modèle non arabe est NLP Town BERT multilingue. Pour les textes avec caractères arabes, l'application utilise MARBERT. Si Hugging Face n'est pas disponible, un fallback local par mots-clés classe le commentaire. Ce fallback est utile, mais limité.

Versión courte : BERT multilingue, MARBERT pour l'arabe, fallback lexical en secours.

Question probable : Le fallback est-il un modèle ML ? Réponse : non, c'est une règle lexicale locale.

Slide 16 – Comparaison des performances

Responsable : Mehdi. **Durée max :** 35 s.

Script : La comparaison est faite sur 15 avis annotés manuellement. Le fallback local obtient 0,53 d'accuracy, car il rate des avis mixtes ou en darija latinisée. Le pipeline principal obtient 0,93 sur ce corpus. Il faut présenter ce résultat comme indicatif.

Versión courte : Les chiffres montrent une tendance sur un petit corpus, pas une validation production.

Question probable : Peut-on généraliser ? Réponse : non, il faudrait un corpus plus large.

Slide 17 – Métriques du pipeline principal

Responsable : Mehdi. **Durée max :** 35 s.

Script : La precision indique si les prédictions d'une classe sont fiables. Le recall indique si on retrouve bien les vrais exemples de cette classe. Le F1-score combine les deux. Ici, les avis neutres sont plus difficiles, car ils mélangent souvent positif et négatif.

Version courte : Precision : qualité des prédictions ; recall : capacité à retrouver les vrais cas ; F1 : équilibre.

Question probable : Pourquoi le neutre est moins bon ? Réponse : les avis neutres sont souvent ambigus.

Slide 18 – DÉMONSTRATION DE L'APPLICATION TASTIFY

Responsable : Mehdi. **Durée max :** 45 s à 1 min.

Script : Cette démonstration montre le parcours principal de Tastify : menu client, commande côté staff, KDS cuisine, puis avis et analyse de sentiment. Si la vidéo se lance correctement, je la laisse tourner brièvement et je commente seulement les étapes clés. Si elle ne se lance pas, je montre les écrans déjà préparés.

Version courte : La démo prouve que les modules communiquent : client, staff, cuisine et avis.

Question probable : Est-ce une vraie démo complète ? Réponse : c'est une démonstration courte de soutenance ; les détails complets peuvent être montrés si le jury le demande.

Slide 19 – Défis rencontrés et solutions

Responsable : Ibtihal. **Durée max :** 35 s.

Script : Les difficultés principales ont été la coordination entre salle et cuisine, le traitement asynchrone, la séparation des rôles et la validation. Les solutions ont été WebSocket avec Redis, Celery, permissions par rôle et tests pytest, Vitest et Playwright.

Version courte : Les défis ont été traités par architecture, rôles, asynchrone et tests.

Question probable : Quel défi a été le plus important ? Réponse : garder la cohérence entre commandes, KDS, paiement et stock.

Slide 20 – Conclusion et perspectives

Responsable : Ibtihal. **Durée max :** 40 s.

Script : Tastify centralise les modules principaux d'un restaurant et ajoute l'exploitation des avis clients. Le projet reste un prototype académique. Les perspectives sont le paiement réel, un corpus IA plus large, les tests de charge, le déploiement production et le monitoring.

Version courte : Le bilan est positif, mais les limites sont clairement identifiées.

Question probable : Que feriez-vous ensuite ? Réponse : paiement réel, données IA plus riches et déploiement production sécurisé.

Slide 21 – Merci / Questions-Réponses

Responsable : Ibtihal. **Durée max :** 15 s.

Script : Merci pour votre attention. Nous sommes prêts à répondre à vos questions.

4 Glossaire technique anti-questions pièges

Terme	Définition simple	Dans Tastify	Erreur à éviter
-------	-------------------	--------------	-----------------

ERP	Système qui centralise plusieurs fonctions d'une organisation.	Tastify est présenté comme prototype académique de mini-ERP restaurant.	Dire que Tastify remplace un ERP commercial complet.
API	Point d'entrée permettant aux applications de communiquer.	Les frontends consomment les API Django REST Framework.	Confondre API et interface visuelle.
REST API	API organisée autour de ressources et méthodes HTTP.	Routes pour plats, tables, commandes, avis, réservations.	Dire que tout passe par WebSocket.
Django	Framework web Python.	Backend, modèles, logique métier.	Dire que Django est le frontend.
DRF	Extension Django pour créer des API REST.	Viewsets, serializers, permissions.	Oublier les permissions côté API.
React	Bibliothèque frontend.	Back-office et portail client.	Dire que React stocke les données métier.
MySQL	Base relationnelle.	Stocke utilisateurs, commandes, avis, paiements.	Dire que Redis remplace MySQL.
Redis	Stockage rapide utilisé comme broker et channel layer.	WebSocket staff et Celery.	Le présenter comme base principale.
Celery	Système de tâches de fond.	Analyse de sentiment et traitements asynchrones.	Dire que Celery est obligatoire pour afficher toutes les pages.
WebSocket	Canal de communication ouvert entre client et serveur.	Mise à jour staff et KDS.	Dire "temps réel" sans expliquer WebSocket/Redis.
JWT	Jeton d'authentification.	Access token et refresh cookie.	Croire que cacher un menu frontend suffit à sécuriser.
KDS	Écran cuisine de suivi des commandes.	Le cuisinier voit et met à jour les statuts.	Le confondre avec une caisse.
NLP	Traitement automatique du langage naturel.	Analyse des commentaires clients.	Dire que NLP veut dire seulement traduction.
Analyse de sentiment	Classification d'un texte en positif, neutre, négatif.	Avis clients analysés et stockés.	Promettre une compréhension parfaite.
Fine-tuning	Adaptation d'un modèle pré-entraîné à une tâche.	Les modèles Hugging Face utilisés sont déjà fine-tunés.	Dire que le fine-tuning a été fait localement.
Fallback	Solution de secours.	Fallback lexical local si Hugging Face manque.	Le vendre comme équivalent au modèle principal.
Accuracy	Part de prédictions correctes.	0,53 fallback, 0,93 pipeline principal sur 15 avis.	Généraliser ces chiffres sans contexte.
Precision	Qualité des prédictions d'une classe.	Utile pour vérifier les faux positifs.	Confondre avec accuracy.
Recall	Capacité à retrouver les vrais cas d'une classe.	Important pour ne pas rater les avis négatifs.	Confondre avec precision.
F1-score	Équilibre entre precision et recall.	Présenté par classe sur le pipeline principal.	Le présenter comme une note générale unique.

5 Démonstration : deux scénarios

5.1 Scénario A : vidéo intégrée

Moment idéal. Slide 18, directement au moment prévu dans la présentation.

Durée idéale. 45 secondes à 1 minute 30. Ne pas dépasser ce temps, sinon la conclusion sera trop rapide.

Avant de lancer. Dire : "Nous allons montrer rapidement le parcours principal : menu client, commande staff, KDS cuisine et avis analysé."

Pendant la vidéo. Parler peu. Commentaire de 20 à 40 secondes : "Ici, on voit le portail client et le back-office. Le serveur crée une commande liée à une table, la cuisine la reçoit dans le KDS, puis le paiement simulé et l'avis client montrent comment les modules communiquent."

Après la vidéo. Dire : "La vidéo montre le flux complet. Si vous le souhaitez, nous pouvons détailler une partie précise en direct."

Si la vidéo ne fonctionne pas. Dire : "Nous avons prévu un plan B : les écrans sont déjà ouverts, je vais commenter le même parcours directement."

5.2 Scénario B : démonstration en direct

Avant la soutenance.

- Docker Desktop ouvert.
- 'start-demo-local.bat' lancé au moins 10 minutes avant.
- Onglets prêts : 'http://localhost:3000', 'http://localhost:3003'.
- Comptes prêts : 'gerant_test', 'serveur_test', 'cuisinier_test', 'client_test', mot de passe 'password123'.
- Notifications coupées, chargeur branché, captures ouvertes en secours.

Pendant la démo.

Action	Phrase exacte	Valeur	Alternative
Portail client/menu	Voici la partie client : consulter les plats, voir les suggestions et préparer une commande.	Expérience client.	Montrer capture client.
Salle serveur	Côté serveur, je sélectionne une table et je crée une commande.	Lien table-commande.	Montrer capture salle.
KDS cuisine	La commande arrive côté cuisine dans le KDS.	Coordination salle-cuisine.	Montrer capture KDS.
Paiement simulé	Le QR mène vers une page de paiement simulé avec token signé.	Flux réaliste sans paiement bancaire.	Expliquer sans scanner.
Avis client	Le client laisse un avis ; l'analyse est faite en tâche de fond.	Sentiment exploité.	Montrer page avis.
Dashboard avis	Le gérant suit les retours et statistiques.	Aide à la décision.	Montrer capture dashboard.

Parcours ultra-court. Salle -> KDS -> avis analysé -> conclusion. Une minute maximum.

6 Cinquante questions-réponses de jury

1. Pourquoi ce domaine ? Parce qu'un restaurant combine commande, cuisine, stock, paiement et avis dans un flux concret.
2. Quelle est la problématique ? Centraliser les opérations et améliorer la coordination salle-cuisine-gestion-client.
3. Qui utilise Tastify ? Gérant, serveur, cuisinier et client.
4. Quelle valeur par rapport à un menu numérique ? Tastify relie plusieurs modules métier.
5. Pourquoi mini-ERP ? Parce qu'il centralise plusieurs fonctions, mais reste un prototype académique.
6. Quelle limite principale ? Paiement réel, production cloud et tests de charge restent des perspectives.
7. Contribution personnelle ? Mehdi : backend, intégration, tests et déploiement. Ibtihaïl : frontends, UX, documentation, UML et validation.
8. Qu'est-ce qui est vraiment développé ? Les modules présents dans le code : menu, tables, commandes, KDS, stock, RH, réservations, paiements, avis, analytics, fidélité, configuration.
9. Pourquoi Django ? Structure, ORM, sécurité, écosystème.
10. Pourquoi DRF ? API REST, serializers, viewsets et permissions.
11. Pourquoi React ? Interfaces interactives séparées et testables.
12. Pourquoi deux frontends ? Les besoins staff et client sont différents.
13. Pourquoi MySQL ? Données relationnelles et cohérence métier.
14. Pourquoi Redis ? WebSocket staff et broker Celery.
15. Pourquoi Celery ? Tâches de fond sans bloquer l'utilisateur.
16. Comment fonctionne le temps réel ? WebSocket avec Django Channels et Redis pour staff/KDS.
17. Comment sécuriser les rôles ? Permissions DRF côté backend ; le frontend ne fait que guider l'affichage.
18. Où sont les routes API ? Dans le routeur Django et les modules spécialisés.
19. Cycle d'une commande ? Table, plats, commande, KDS, préparation, service, paiement simulé.
20. Quand le stock diminue ? Selon les recettes et mouvements de stock.
21. Si un ingrédient manque ? Le gérant voit les seuils et les disponibilités sont contrôlées.
22. KDS ? Écran cuisine pour suivre et changer les statuts.
23. Réservation ? Création client, disponibilité, confirmation ou annulation staff.
24. Paiement réel ? Non, simulation avec logique de réconciliation.
25. QR paiement ? Token signé temporaire lié à table et commande.
26. Fidélité ? Profil client, points, transactions et récompenses.
27. Avis ? Commentaires analysés et visibles côté gérant.
28. Démo à montrer ? Menu, salle, KDS, paiement simulé, avis.
29. Analyse de sentiment ? Classification d'un avis en positif, neutre ou négatif.
30. Entrée du modèle ? Le commentaire texte de l'avis.

31. Avez-vous entraîné BERT/MARBERT ? Non, modèles pré-entraînés/fine-tunés via Hugging Face.
32. Pourquoi BERT ? Adapté au contexte des avis courts multilingues.
33. Pourquoi MARBERT ? Adapté aux textes arabes.
34. Choix modèle ? Détection simple de caractères arabes.
35. Sans clé Hugging Face ? Fallback lexical local.
36. Pourquoi fallback 0,53 ? Mots-clés limités, avis mixtes difficiles.
37. Pourquoi pipeline 0,93 ? Résultat indicatif sur 15 avis annotés.
38. Precision vs recall ? Precision : qualité des prédictions ; recall : vrais cas retrouvés.
39. Pourquoi neutre difficile ? Les avis neutres mélangent souvent positif et négatif.
40. Pourquoi Deep Learning via API ? Les modèles exécutés sont des modèles Deep Learning ; l'intégration est applicative.
41. Lancement démo ? Docker Desktop puis 'start-demo-local.bat'.
42. Comptes démo ? 'gerant_test', 'serveur_test', 'cuisinier_test', 'client_test'.
43. Tests ? pytest, Vitest, Playwright.
44. Couverture 93 % ? Couverture backend indiquée dans le rapport.
45. Si Celery tombe ? Les tâches de fond sont retardées.
46. Si Redis tombe ? WebSocket et broker Celery sont impactés.
47. Difficultés ? Rôles, KDS, cohérence commandes/paiements/stocks, sentiment.
48. Pourquoi Docker Compose ? Reproductibilité de l'environnement complet.
49. Manque pour production ? HTTPS, monitoring, sauvegardes, secrets, tests de charge, paiement réel.
50. Un mois de plus ? Paiement réel, corpus IA plus large, tests de charge et déploiement sécurisé.

7 Questions personnelles pour chaque étudiant

Questions possibles à Mehdi

1. Explique le backend Django et le découpage en apps. Référence : 'app/backend/apps'.
2. Explique le routeur API. Référence : 'tastify_backend/api_router.py'.
3. Explique JWT et permissions. Référence : 'apps/users'.
4. Explique Redis et WebSocket. Référence : 'core/consumers.py', 'core/routing.py'.
5. Explique Celery et l'analyse de sentiment. Référence : 'apps/avis/tasks.py'.
6. Explique le fallback local. Référence : 'apps/avis/sentiment_service.py'.
7. Explique le paiement QR. Référence : 'apps/paiements/tokens.py'.
8. Explique Docker Compose. Référence : 'docker-compose.yml'.
9. Explique les tests backend. Référence : chapitre tests du rapport.
10. Explique une limite technique importante. Réponse attendue : paiement réel, production, charge, corpus IA.

Questions possibles à Ibtihal

1. Explique le besoin métier. Référence : chapitre contexte.
2. Explique les acteurs. Référence : gérant, serveur, cuisinier, client.
3. Explique pourquoi deux interfaces. Référence : frontends client et staff.
4. Explique le parcours client. Référence : menu, réservation, paiement, fidélité, avis.
5. Explique le parcours staff. Référence : dashboard, salle, KDS, stock, RH.
6. Explique les diagrammes UML. Référence : chapitre conception.
7. Explique l'étude de l'existant. Référence : Lightspeed, Toast, Revel.
8. Explique la conclusion et les perspectives. Référence : slide 17.
9. Explique la validation fonctionnelle. Référence : tests Playwright et captures.
10. Explique les limites à dire honnêtement. Réponse attendue : prototype académique, paiement simulé, IA indicative.

8 Simulation de soutenance

Temps	Qui	À dire	À raccourcir si besoin
0–1 min	Ibtihal	Titre, plan, contexte restaurant.	Détails du plan.
1–2 min	Ibtihal	Problématique et existant.	Tableau comparatif.
2–4 min	Ibtihal puis Mehdi	Solution, deux interfaces, architecture.	Exemples secondaires.
4–6 min	Mehdi	Commande vers KDS, traitement avis, technologies.	Liste complète des outils.
6–8 min	Mehdi	Pipeline sentiment, modèles, métriques.	Théorie ML/DL.
8–9 min	Mehdi	Slide 18 : vidéo ou démo très courte.	Garder seulement menu -> KDS -> avis.
9–10 min	Ibtihal	Slides 19 à 21 : défis, limites, conclusion et ouverture questions.	Ne jamais supprimer les limites.

Version 8 minutes. Supprimer les détails de l'étude de l'existant, passer les slides 7 et 11 en une phrase, réduire ML/DL à une phrase, présenter seulement une métrique globale et faire une démo de 30 à 45 secondes.

9 Check-list de la veille et du jour J

Ce soir. Relire les scripts, répéter deux fois, tester transitions, vérifier présentation, vidéo, polices, liens, captures, comptes de test, chargeur, adaptateurs et clé USB.

Demain matin. Ordinateur chargé, Docker Desktop lancé si démo live, présentation ouverte, vidéo ouverte localement, notifications coupées, navigateur prêt, chargeur branché.

10 erreurs à éviter. Lire les slides, dépasser 10 minutes, promettre un paiement réel, dire que BERT a été entraîné localement, généraliser les métriques IA, oublier Redis/Celery, minimiser les limites, faire une démo longue, se contredire sur les rôles, regarder seulement l'écran.

10 Évaluation comme un jury

Critère	Points	Amélioration rapide
Maîtrise du sujet	2	Savoir expliquer le besoin en une phrase.
Cohérence rapport / présentation / code	2	Garder les mêmes termes : Django, React, MySQL, Redis, Celery.
Expression orale	2	Phrases courtes, pas de lecture.
Répartition entre étudiants	2	Blocs cohérents, transitions exactes.
Précision technique	2	Expliquer API, WebSocket, Celery, fallback.
Démonstration	2	Vidéo prête et démo live courte en secours.
Gestion du temps	2	Chronométrer deux répétitions.
Réponses aux questions	2	Réviser les 10 questions dangereuses.
Honnêteté sur les limites	2	Dire clairement paiement simulé et corpus réduit.
Qualité visuelle	2	Ouvrir la présentation avant, vérifier affichage et vidéo.

Objectif réaliste. Viser 16/20 ou plus en étant clair, honnête, rapide et cohérent.